

Codeman-HUD

Overview

The **Codeman-HUD** is a modular Head-Up Display designed to allow players to interact with the game environment and safely execute custom scripting logic. By utilizing a **Linkset-wide communication model (LINK_SET)**, the HUD eliminates ownership permission conflicts and prevents screen-blocking issues.



The HUD consists of three core buttons linked together:

1. Script Button (Root Prim): The central hub and data controller (Server) of the HUD.

- **Responsibilities:**

- Delivers the code template (TemplateScript) to the player's inventory.
- Allowing players to simply drag-and-drop their compiled SlowScript anywhere on the HUD without entering edit modes.
- Hosts the player's script locally and broadcasts the final triggers (e.g., SLOW command) to the maze ghosts via a dedicated region channel.

```

string TEMPLATE_NAME = "TemplateScript";
string SLOW_SCRIPT = "SlowScript";
integer GHOST_CHANNEL = 777;

default
{
    state_entry()
    {
        llAllowInventoryDrop(TRUE);
    }

    touch_start(integer total_number)
    {
        key user = llDetectedKey(0);
        llOwnerSay("Giving you the Slow Logic Template.");
        llGiveInventory(user, TEMPLATE_NAME);
    }

    link_message(integer sender_num, integer num, string str, key id)
    {
        if (str == "CHECK_AND_RUN")
        {
            if (llGetInventoryType(SLOW_SCRIPT) == INVENTORY_SCRIPT)
            {
                llOwnerSay("Script found! Executing player logic...");
                llMessageLinked(LINK_SET, 100, "EXECUTE_USER_LOGIC", NULL_KEY);
            }
            else
            {
                llOwnerSay("Error: 'SlowScript' not found. Please drag and drop your script onto the HUD first!");
            }
        }
        else if (str == "SLOW_ON")
        {
            llRegionSay(GHOST_CHANNEL, "SLOW");
            llOwnerSay("Broadcasted SLOW command to Ghosts.");
        }
    }
}

```

2. Run Button (Child Prim) The execution trigger (Client).

- **Responsibilities:**

- When clicked, checks and broadcasts a signal (CHECK_AND_RUN).

```

0 default
1 {
2     touch_start(integer total_number)
3     {
4         llMessageLinked(LINK_SET, 0, "CHECK_AND_RUN", NULL_KEY);
5     }
6 }

```

3. Exit Button (Child Prim): The session terminator and cleanup utility.

- **Responsibilities:**

- Broadcasts a termination signal (STOP_GAME) to clear the active maze session and reset ghost behaviors.
- Forces the release of the player's camera controls by resetting tracking parameters (llClearCameraParams), safely returning the view to the default third-person perspective.

```

integer PACMAN_CHANNEL = -99;

default
{
    touch_start(integer total_number)
    {
        llOwnerSay("Exit the maze");
        llRegionSay(PACMAN_CHANNEL, "DIE_PACMAN");
        llSetCameraParams([CAMERA_ACTIVE, FALSE]);
        llClearCameraParams();
    }
}

```

Workflow Summary

1. **Fetch:** Player clicks **Script Button** -> Receives Template.
2. **Inject:** Player edits code -> Drags SlowScript onto the HUD-> Stored in **Root (Script Button)**.
3. **Execute:** Player clicks **Run Button** -> Sends Link Message ->**Root** validates and executes the custom logic -> Ghosts slow down.
4. **Terminate:** Player clicks **Exit Button** -> Camera resets -> Maze clea